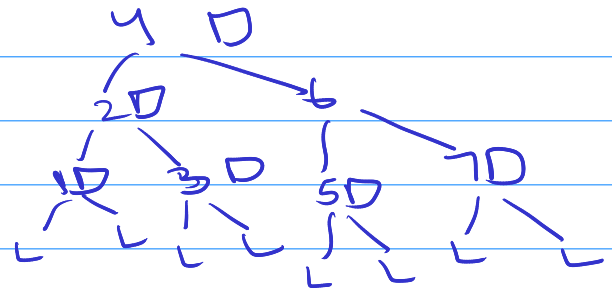
Binary Tree to flat array
Diagram explanation with structural induction

- □ → not yet processed
- □ → currently being processed (current call)
- □ → fully processed (returned)
- □ → waiting (call in progress but paused)

- initial state
  call stack empty
  accumulator [ ]

Tree

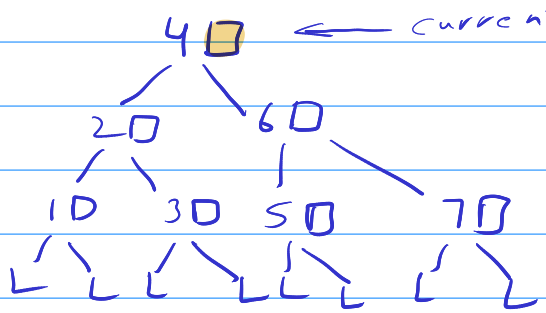


Call 1: flatten(node 4, [ ]) — PUSH

call stack:

flatten(node 4, [ ]) ← current

Tree:



← current (tag: Tree.N, V: 4)

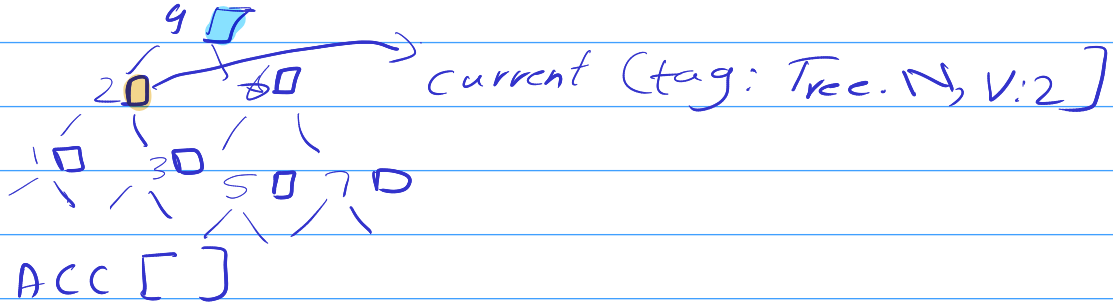
Accumulator: [ ]

action: Enter node case. call flatten (node 2, [ ])
→ push call 2.

Call2: flatten (node2, [ ]) – Push

flatten(node4, [ ]) ← waiting
flatten(node2, [ ]) ← current

Tree:



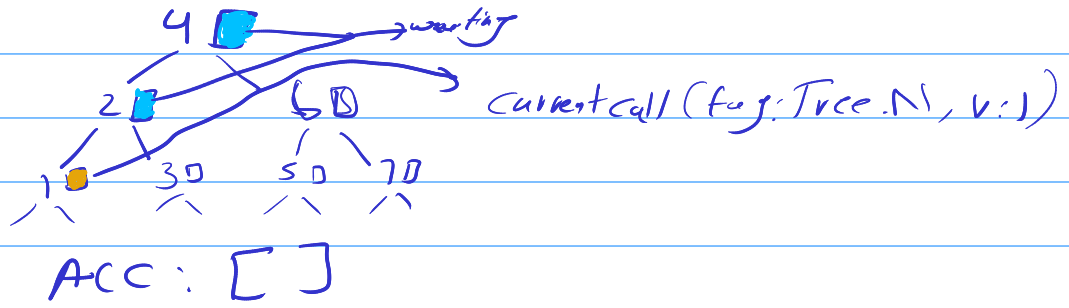
Action: Enter node case. Call (node1, [ ]) → Push call3

Call3: flatten (node1, [ ]) – Push

call stack:

flatten(node4, [ ])
flatten(node2, [ ])
flatten(node1, [ ])

Tree:



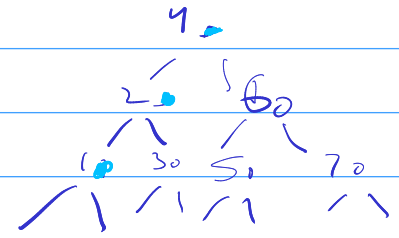
Action: Enter node case. call flatten(leaf, [ ])
↘ Push call 4

call4: flatten (leaf, [ ]) – Push

Call stack:

flatten(node 4, [ ]) ← waiting
flatten(node 2, [ ]) — waiting
flatten(node 1, [ ]) — waiting
flatten(leaf, [ ]) — current

Tree:



L. ⟶ current (tag : Tree . L)

ACC [ ]

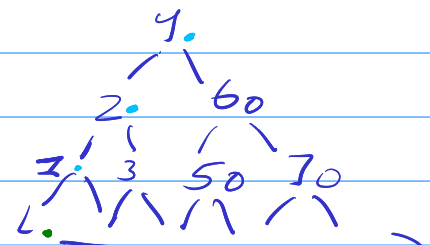
Action: Leaf case, Return [ ] ⟶ POP Call 4.

Call 4 - POP

Call Stack:

flatten(node 4, [ ]) — waiting
flatten(node 2, [ ]) — waiting
flatten(node 1, [ ]) — (received [ ])

Tree:



ACC [ ]

calls continues: left = [ ], left.push(1) → left = [1]

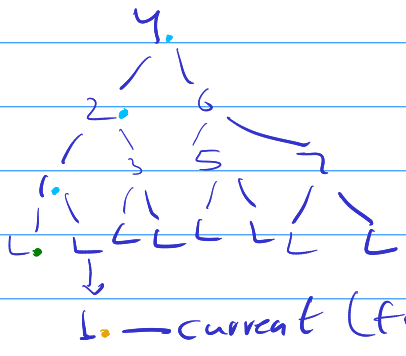
⇒ Call right leaf node ⇒ flatten(leaf, [1]) → Push ca 115

Call 5 : flatten(leaf, [1]) — push

call stack:

flatten(node4, [1]) — waiting
flatten(node2, [1]) — waiting
flatten(node1, [1]) — waiting
flatten(leaf, [1]) — current

Tree:



1 — current (key: Tree.L)

ACC : [1]

Action: leaf case. Return [1] → POP call5

call5 — POP

Return [1] to call3

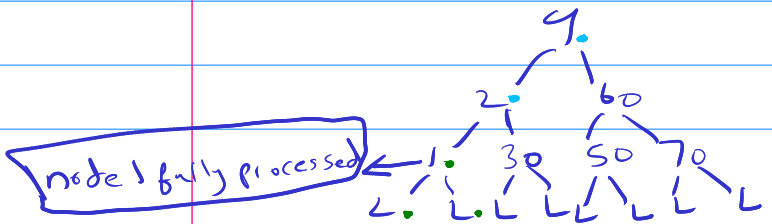
call3 returns [1] → POP call3.

call1 — POP

call stack:

flatten(node4, [1]) — waiting
flatten(node2, [1]) — current (received [1])

Tree:



ACC [1]

call 2 continues: left = [], left.push(2)

⇒ left = [1,2], call flatten(node3, [1,2]) → Push call 6.

↓ right child of node 2.

Call 6: flatten(node3, [1,2]) — Push

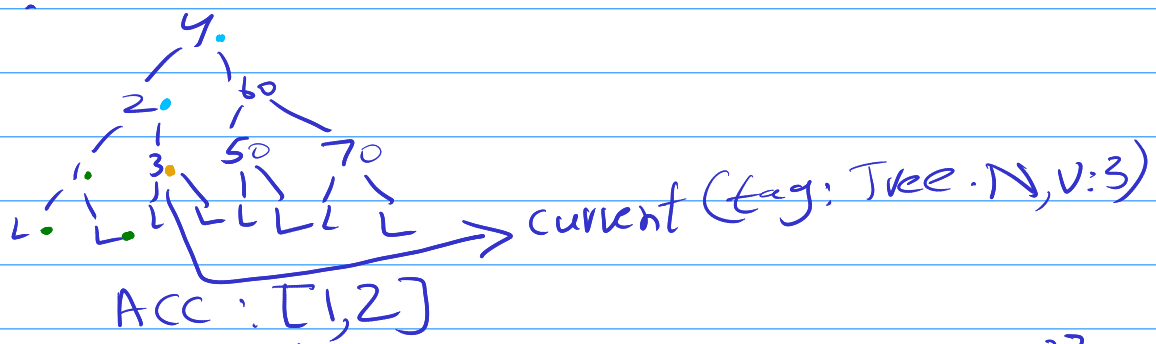
Call stack:

flatten(node4, []) — waiting

flatten(node2, []) — waiting

flatten(node3, [1,2]) — current

Tree:



Action: Enter node case. Call flatten(leaf, [1,2])

↓ [left child of node3]

Push call 7

Call 7: flatten(leaf, [1,2]) — Push

call stack:

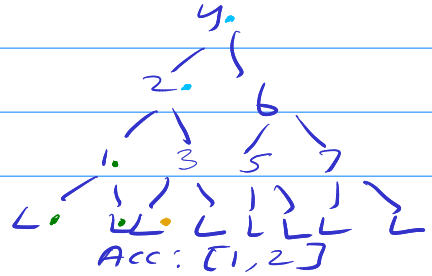
flatten(node4, []) — waiting

flatten(node2, []) — waiting

flatten(node3, [1,2]) — waiting

flatten(leaf, [1,2]) → current

Tree:



Action: Leaf case. Return [1,2] → Pop call 7

call7 — pop

Return [1,2] to call6.

call continues: left = [1,2], left.push(3)

⇒ left = [1,2,3], call flatten(leaf, [1,2,3]) → push call8

call8: flatten(leaf, [1,2,3]) — Push (right child)

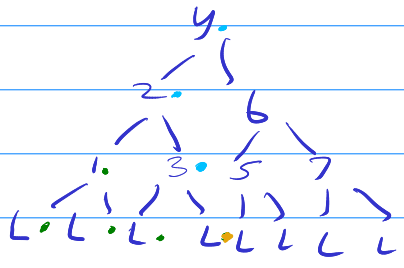
Call Stack:

flatten(node4, [ ]) — waiting

flatten(node2, [ ]) — waiting

flatten(node3, [1,2]) — waiting

flatten(leaf, [1,2,3]) — current



current t (tag: Tree.L)

Acc: [1,2,3]

Action: leaf case. Return [1,2,3]

Pop call8.

call8 — pop

Return [1,2,3] to call6.

call6 returns [1,2,3] → pop call6.

call6 — pop

Return [1,2,3] to call2.

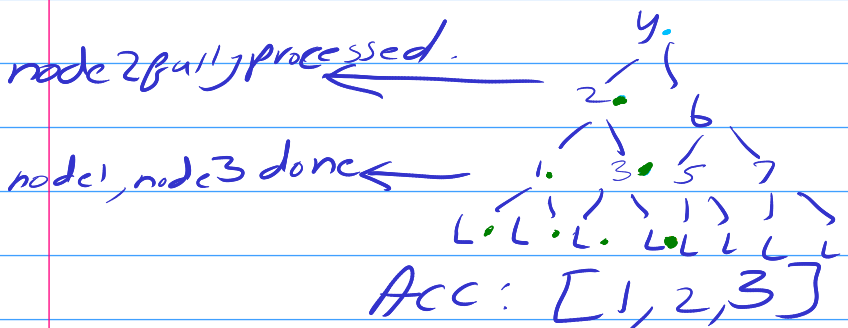
call2 returns [1,2,3] → pop call2.

call2 — pop

call stack:

flatten(node4, [ ]) → current (received [1,2,3])

Tree:



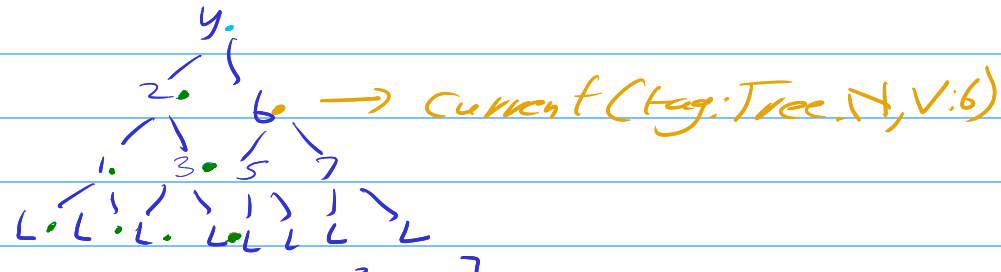
call 1 continues: left = [1, 2, 3], left.push(4)
=> left = [1, 2, 3, 4]
call right child of node 4 flatten(node6, [1, 2, 3, 4])
↓ Push call 9.

call 9: flatten(node6, [1, 2, 3, 4]) — Push

call stack:

flatten(node4, []) — waiting
flatten(node6, [1, 2, 3, 4]) — current

Tree:

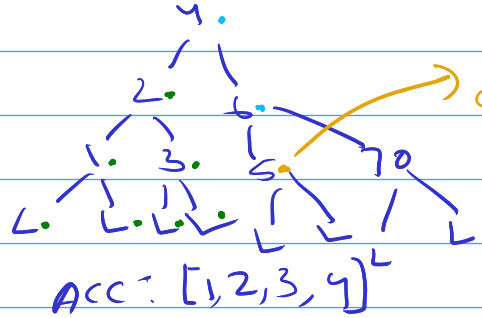


Action: Enter node case
=> call flatten (node5, [1, 2, 3, 4]) — Push call 10.
call 10: flatten(node5, [1, 2, 3, 4]) — Push

call stack:

flatten (node4, []) — waiting
flatten (node6, [1, 2, 3, 4]) — waiting
flatten (node5, [1, 2, 3, 4]) — current

Tree:



ACC: [1,2,3,4]

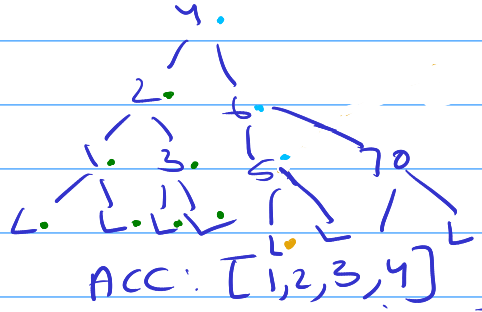
Action: Enter node case. Call flatten(leaf, [1,2,3,4]) ↓Push call 11.

call: flatten(leaf, [1,2,3,4]) - Push [left child]

Call stack:

flatten(node4, [ ])	waiting
flatten(node6, [1,2,3,4])	waiting
flatten(node5, [1,2,3,4])	waiting
flatten(leaf, [1,2,3,4])	current

Tree:



ACC: [1,2,3,4]

Action: Leaf case. Return [1,2,3,4] - POP call 11

Call 11 - Pop

Return [1,2,3,4] to call 10.

call 10 continues: left = [1,2,3,4]

left.push(5) → left = [1,2,3,4,5]

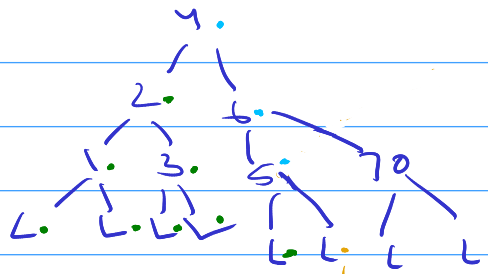
Call flatten(leaf, [1,2,3,4,5]) → Push call 12.

call 12: flatten(leaf, [1,2,3,4,5]) - Push

Call stack:

flatten(node4, [])	— waiting
flatten(node6, [1,2,3,4])	— waiting
flatten(node5, [1,2,3,4])	— waiting
flatten(leaf, [1,2,3,4,5])	— current

Tree:



ACC: [1,2,3,4,5]

Action: Leaf case. Return [1,2,3,4,5]

Pop call 12 (Right child)

call 12 - Pop

Return [1,2,3,4,5] to call 10.

call 10 returns [1,2,3,4,5] → Pop call 10.

call 10 - Pop

Return [1,2,3,4,5] to call 9

call 9 continues: left = [1,2,3,4,5]

left.push(6) → left = [1,2,3,4,5,6]

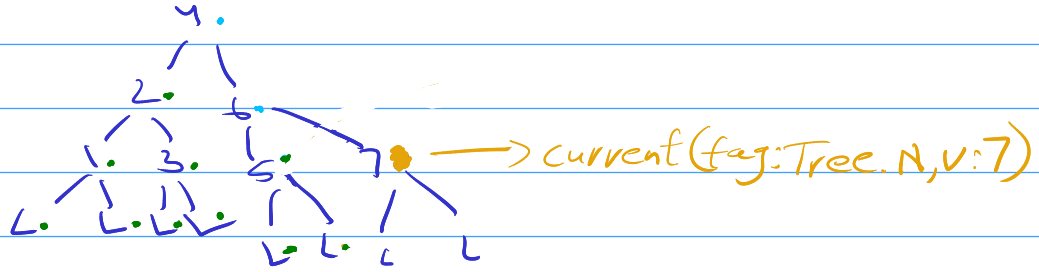
call flatten(node7, [1,2,3,4,5,6]) → Push call 13

call 13: flatten(node7, [1,2,3,4,5,6]) — Push

call stack:

flatten(node4, [])	— waiting
flatten(node6, [1,2,3,4])	— waiting
flatten(node7, [1,2,3,4,5,6])	— current

Tree:



ACC: [1, 2, 3, 4, 5, 6]

Action: Enter node case:

call flatten (leaf, [1, 2, 3, 4, 5, 6]) → Push call 14.

call 14: flatten (leaf, [1, 2, 3, 4, 5, 6]) - Push

call 14: flatten (leaf, [1, 2, 3, 4, 5, 6]) - Push

Call stack:

flatten (node 4, [])	waiting
flatten (node 6, [1, 2, 3, 4])	waiting
flatten (node 7, [1, 2, 3, 4, 5, 6])	waiting
flatten (leaf, [1, 2, 3, 4, 5, 6])	current

Tree:



ACC: [1, 2, 3, 4, 5, 6]

Action: leaf case. Return [1, 2, 3, 4, 5, 6]

↓ Pop call 14.

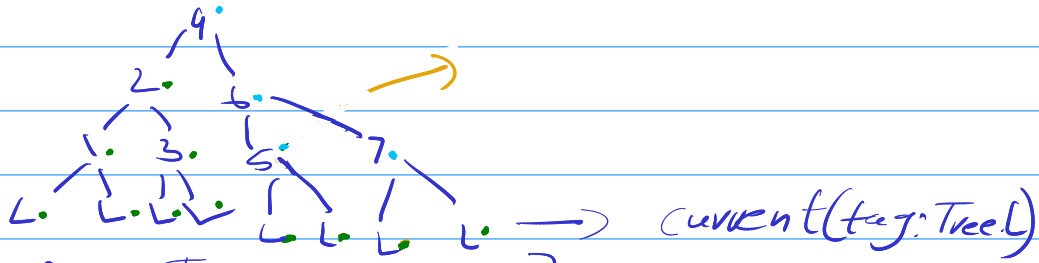
call 14 — pop
Return [1,2,3,4,5,6) to call 13.
call 13 continues: left = [1,2,3,4,5,6)
left.push(7) → left = [1,2,3,4,5,6,7]
call flatten(leaf, [1,2,3,4,5,6,7]) — push call 15.

call 15: flatten(leaf, [1,2,3,4,5,6,7]) — push

Call stack:

flatten(node 9, [])	waiting
flatten(node 6, [1,2,3,4])	waiting
flatten(node 7, [1,2,3,4,5,6])	waiting
flatten(leaf, [1,2,3,4,5,6,7])	current

Tree:



Acc: [1,2,3,4,5,6,7]
Action: leaf case. Return [1,2,3,4,5,6,7]
↓ pop call 15.

call 15 — pop
Return [1,2,3,4,5,6,7) to call 13
call 13 returns [1,2,3,4,5,6,7] — pop call 13.

call 13 — pop
Return [1,2,3,4,5,6,7] to call 9.
call 9 returns [1,2,3,4,5,6,7] — pop call 9.

call 9 — pop
Return [1,2,3,4,5,6,7] to call 1.
call 1 returns [1,2,3,4,5,6,7] → pop call 1.

final state
call stack: (empty)

Tree:

